



Project Proposal

CUDA-Accelerated Integration for High-Throughput Blockchain Networks

Rayyan Hassan Salman (2023601)

Ahmad Ibrahim Ahmed (2023066)

Faculty of Computer Science and Engineering

Ghulam Ishaq Khan Institute of Engineering Sciences and Technology (GIKI)

Course Instructor:

Dr. Taj Muhammad Khan

April 2026

1 Scope and Motivation

Why this project? While Proof-of-Work mining has long been hardware-accelerated, the true scalability bottleneck in legacy networks (like the Ethereum Virtual Machine) is their reliance on sequential CPU execution for transaction validation and state updates. This creates a hard limit on Transactions Per Second (TPS). This project addresses this industry-wide bottleneck by transitioning the transaction validation pipeline—specifically cryptographic signature verification and state-tree generation—from a sequential CPU model to a massively parallel GPU environment.

What it does: To prove this parallel execution concept, we will profile a representative sequential base node (evaluating educational frameworks like Dave Nash’s `TestChain` [C++] or Daniel van Flymen’s `blockchain` [Python]). We will replace its core sequential validation loop with a custom CUDA acceleration layer, processing massive batches of ECDSA signatures and generating the block’s Merkle Root concurrently.

Where can it be used and to what benefit? This architecture mirrors the parallel execution engines of next-generation networks like Solana and Monad. The primary benefit is demonstrating that hardware-accelerating the transaction validation pipeline can reduce block processing latency from seconds to milliseconds, drastically scaling network throughput without altering the core consensus protocol.

2 Overall Structure and Working

The system relies on a Host-Accelerator architecture. The baseline blockchain acts as the host, and our custom CUDA library acts as the parallel execution engine.

The GPU Part (>60% of work): The custom CUDA kernels will replace the heaviest CPU loops found in the base codebase.

- *Batch ECDSA Verifier:* An embarrassingly parallel kernel designed to verify thousands of digital signatures concurrently (1 Thread = 1 Signature payload), breaking the sequential validation bottleneck.
- *Parallel Merkle Root Generator:* A shared-memory reduction kernel that takes verified transaction hashes from the mempool and recursively folds them until the Merkle Root is formed.
- *PoW Miner:* A highly parallelized SHA-256 brute-force engine utilizing millions of threads

to find valid cryptographic nonces for block finalization.

The Non-GPU Part: This comprises the base framework and our critical integration layer.

- *Base Framework:* Handles network state, the transaction mempool, and block logic.
- *Wrapper & Interfacing:* We will develop secure cross-language bindings to safely marshal transaction batches from the host’s memory space into the PCIe bus, triggering the CUDA kernels, and returning the computed state back to the CPU.

3 Contribution & Implementation

Implementation Plan: (1) Fork the selected sequential repository (`TestChain` or `blockchain`) and establish a CPU execution baseline; (2) Develop the isolated CUDA kernels for Parallel Verification and Merkle Trees; (3) Write the memory-safe wrapper integration to inject our custom parallel drivers directly into the host’s block-generation pipeline.

Techniques from the Book:

- *Constant Memory:* Storing elliptic curve parameters and SHA-256 constants (K array) for zero-latency broadcast reads.
- *Shared Memory:* Utilized heavily to keep transaction hashing entirely on-chip.
- *Parallel Reduction Algorithms:* Applying logarithmic time-complexity reduction ($O(\log N)$) to collapse the Merkle Tree, utilizing advanced shared-memory thread synchronization.

Bonus: Advanced Concepts (Not in Class): To exceed the standard curriculum, we will implement the following advanced optimizations:

- *Foreign Function Interfaces (FFI / Bindings):* Engineering a memory-safe cross-language bridge (via `pybind11` or `ctypes`) to allow a high-level host framework (e.g., Python) to natively execute and marshal data to our compiled low-level CUDA drivers.
- *Read-Only Data Cache (`__restrict__`):* Utilizing compiler-level pointer aliasing hints when bridging the host framework to the GPU, forcing the hardware to route massive signature reads through the ultra-fast texture cache instead of global memory.

- *CUDA Dynamic Parallelism (CDP)*: Engineering the GPU to autonomously spawn child grids during the recursive Merkle Tree generation, eliminating the latency of repeated CPU-to-GPU kernel launch overheads from the host framework.
- *CUDA Streams (Asynchronous Execution)*: Overlapping the host framework's state management with the GPU-side cryptographic execution, ensuring neither processor sits idle while data batches move across the PCIe bus.

Correctness Verification: The node will run in a "Shadow Mode" during testing. Both the original sequential functions of the repository and our parallel GPU kernels will process the same batches. The system will assert that the GPU-generated out-

puts match the host CPU outputs exactly.

Speed Measurement: Using the `cudaEventRecord` API, we will benchmark the original serial framework's validation time against our accelerated parallel node, graphing the execution curves for transaction batching.

4 GitHub Repository

- **Repo Link:** <https://github.com/rayy1704/cuda-accelerated-blockchain>

We commit to regular pushes (multiple commits per week) demonstrating the fork of the base repository, kernel development, and the interfacing layer. This proposal's details will be integrated into the `README.md`.